SQL Joins

A **JOIN** combines rows from two or more tables based on a related column. The *match condition* (the `ON` clause) is the bridge between tables.

Join Type	What rows are kept
`CROSS JOIN`	Every combination of rows – no `ON` clause. $\|A\| \times \|B\|$ rows.
`INNER JOIN ...ON`	Only rows where the `ON` condition matches in *both* tables.
`LEFT (OUTER) JOIN`	All rows from the left table; `NULL`s where no right-side match.
`RIGHT (OUTER) JOIN`	All rows from the right table; `NULL`s where no left-side match.
`FULL OUTER JOIN`	All rows from both tables; `NULL`s wherever no match exists.

Key insight: `FULL OUTER JOIN` = `LEFT JOIN` ∪ `RIGHT JOIN`. Omitting `ON` in an old-style `FROM a, b` query is an implicit cross join, dangerous on large tables!

Relational Algebra – Quick Reference

Each operator takes one or two relations and returns a new relation. SQL is the practical implementation; understanding the algebra helps you reason about and optimize queries.

Op	Name (arity)	What it does / SQL equivalent
σ	Selection (unary)	Keep tuples satisfying a condition. **SQL:** `WHERE`.
π	Projection (unary)	Keep only listed attributes; duplicates removed. **SQL:** `SELECT DISTINCT`.
ρ	Rename (unary)	Rename relation and/or attributes. Needed when joining a table with itself.
$\cup$	Union (binary)	All tuples in R or S (union-compatible required). **SQL:** `UNION`.
$\cap$	Intersection (bin.)	Tuples in both R and S. **SQL:** `INTERSECT`.
$-$	Difference (bin.)	Tuples in R but not S. NOT symmetric. **SQL:** `EXCEPT`.
$\times$	Cross product (bin.)	Every pair of tuples from R and S.
$\bowtie$	(Natural) Join (bin.)	Cross product + matching on equal attribute values.

Union compatibility (required for $\cup$, $\cap$, $-$): same number of attributes with compatible types. Names do *not* need to match, the result inherits names from the *left* relation.

Selection & Projection Properties

Property	Explanation
$\sigma_{c_1}(\sigma_{c_2}(R)) = \sigma_{c_2}(\sigma_{c_1}(R))$	Selection is **commutative**.
$\sigma_{c_1 \wedge c_2}(R) = \sigma_{c_1}(\sigma_{c_2}(R))$	Conjunction splits into nested selections.
$\pi_A(\pi_B(R)) = \pi_A(R)$ if $A \subseteq B$	Cascade rule for projection.
π removes duplicates; SQL `SELECT` keeps them.	Use `SELECT DISTINCT` to match π.

Schema Used on Side B (Spotify example)

`Songs(song_id, title, artist)` `Plays(play_id, user_id, song_id, played_at)` `Students(sid, name, major, gpa)`

Songs

song_id	title	artist
1	Blinding Lights	The Weeknd
2	Levitating	Dua Lipa
3	Peaches	Justin Bieber

Plays

play_id	user_id	song_id	played_at
1	u1	1	2024-01-01
2	u1	1	2024-01-02
3	u2	2	2024-01-01

Students

sid	name	gpa
1	Anya	3.8
2	Boris	3.2
3	Carmen	3.9

*Use Side A's schema and sample data. Write actual SQL or relational algebra – not pseudocode. Boxes marked **(discuss)** are debriefed with the class after individual work.*

Box 1 Join Types, Step by Step *(8 min, discuss)*

Goal: understand which rows each join type keeps, using `Songs` and `Plays`.

(a) Write a query using `INNER JOIN` to return each `play_id` alongside the `title` of the song that was played. How many rows does the result contain, and why?

(b) Replace `INNER JOIN` with `LEFT JOIN` (keeping `Songs` on the left). Which song(s) now appear with a `NULL` `play_id`, and why?

(c) A classmate writes `FROM Songs, Plays` with no `WHERE` condition. How many rows does this produce, and what join type does it correspond to?

.....

.....

Box 2 Relational Algebra – Selection & Projection *(5 min)*

Write the relational algebra expression (using σ, π) for each task.

(a) All attributes of songs whose artist is `'Dua Lipa'`.

(b) Just the `title` of every song whose artist is `'Dua Lipa'`. Write this *two* ways: π outside σ, then σ outside π, does the second always work?

(c) A classmate says "π and SQL `SELECT` are identical." In one sentence, explain why they are not.

.....

Box 3 Set Operations *(5 min)*

Use the **Students** table (sid, name, major, gpa) from Lecture 5. Let $R = \pi_{\texttt{name}}(\sigma_{\texttt{major='CS'}}(\textit{Students}))$ and $S = \pi_{\texttt{name}}(\sigma_{\texttt{gpa}>3.5}(\textit{Students}))$.

(a) List the tuples in R and S using the Lecture 5 sample data. Then give $R \cup S$, $R \cap S$, and $R - S$.

(b) Write the SQL equivalents for $R \cup S$ and $R - S$ (use `UNION` and `EXCEPT`).

(c) A classmate tries $\pi_{\texttt{name}}(\textit{Students}) \cup \pi_{\texttt{title}}(\textit{Songs})$. Is this valid? Why or why not?

. . . . . .

Box 4 Rename & Self-Join *(6 min, discuss)*

(a) Explain in one sentence *why* the rename operator ρ is needed when joining a relation with itself.

. . . . . .

(b) Write a relational algebra expression that finds all *pairs* of students (by name) in the **same major**. Each pair (a, b) should appear only once – use $<$ on `sid` to avoid duplicates. *Hint: rename Students to S1 and S2.*

(c) Translate your expression from (b) into SQL.

Box 5 Putting It Together *(8 min, discuss)*

(a) *Never-played songs.* Using relational algebra, find the titles of songs that appear in `Songs` but have *never* appeared in `Plays`. Use set difference.

(b) Write the SQL version of (a) using `LEFT JOIN ... WHERE ... IS NULL`.

(c) *Reflect.* Part (b) could also be written with `NOT IN` or `NOT EXISTS`. Sketch one alternative in a single line (full SQL not required).

.....

.....